Face Recognition Using the Eigenfaces Method

Image Processing & Recognition — Exercise (Part II)
Python implementation (NumPy, Pillow, OpenCV, Matplotlib)

1. Aim

The goal of this exercise is to build a face recognition system based on the Eigenfaces method and to evaluate the quality of its recognition. The original task was provided as the MATLAB scripts *trainingEF.m* and *testingEF.m*; here the same algorithm is re-implemented in Python. The system is trained on a set of face images, tested on several faces from the collection, extended with an extra face produced by an AI face generator, and finally tested in real time on a face captured from a webcam.

2. Method

Every image is a 300×300 grayscale picture, which is treated as a single point in a 90,000-dimensional space (one dimension per pixel). All training faces are stacked as columns of a matrix and their average, the *mean face*, is subtracted from each column so that only the differences between faces remain.

The principal directions in which faces differ from the average are the *eigenfaces*. Computing them directly would require the 90,000×90,000 covariance matrix, which is too large. Instead the small matrix $L = A^T \cdot A$ of size M×M is used (M is the number of training images), and its eigenvectors V are mapped back to the image space with $U = A \cdot V$. Each training face is then described by its projection onto these eigenfaces.

To recognise a new face, it is centered with the same mean face, projected onto the eigenfaces, and compared with every training projection using the Euclidean distance. The training face with the smallest distance is returned as the match. The distances are unnormalised, so only their relative order matters — the smallest value wins.

3. Data and Preprocessing

The provided training set contains seven frontal faces (beard, brownwoman, chinawoman, eurowoman, girl, girl2, spanishwoman), each a 300×300 grayscale *.pgm* image. The test set contains similar images with a *_t* suffix. An additional face was downloaded from an AI face generator (generated.photos): one image was used for training and a second, similar one for testing. Both were center-cropped to a square, converted to grayscale and resized to 300×300 before being saved as *.pgm*, so that they match the format of the original data. The webcam face was captured with OpenCV and stored in the same way.

4. Results

4.1 Baseline test (7 training faces)

Training on the seven provided faces and testing with *whitewoman_t.pgm* reproduced the reference MATLAB result exactly: the face was matched to *eurowoman.pgm* with a minimum distance of 0.1004 (×1e9), identical to the distance vector shown in the course material. As there is no ‘whitewoman’ in the training set, eurowoman is the closest available face, which is a reasonable nearest match.

Figure 1. Baseline test: whitewoman_t is matched to the most similar training face, eurowoman.

The remaining collection faces were then tested against the same seven training faces. Table 1 summarises the results.

Test image	Recognised as	Min. distance (x1e9)	Result
girl_t	girl	0.1124	Correct
spanishwoman_t	spanishwoman	0.2031	Correct
brownwoman_t	girl2	0.1982	Wrong (brownwoman = 0.2292)
chinat_t	spanishwoman	0.2777	Wrong (chinawoman = 0.3011)
bread_t	beard	0.0078	Non-face input

Table 1. Recognition with seven training faces.

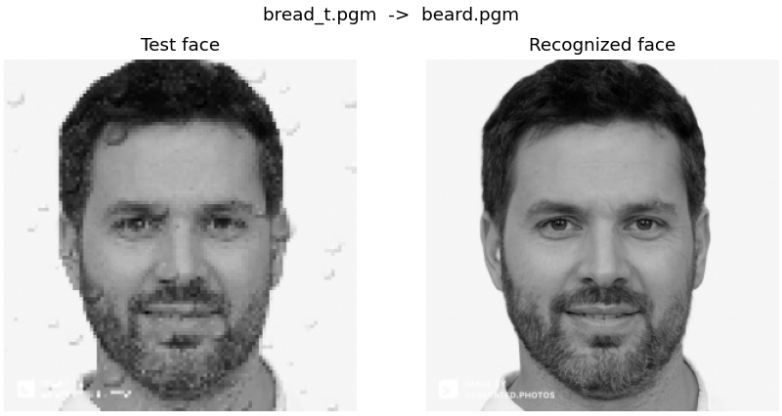

Figure 2. bread_t is not a face, yet the system still returns its nearest neighbour (beard) with a very small distance.

4.2 Adding an AI-generated face (8 training faces)

A new face produced by an AI generator was added to the training set, raising it to eight faces, and the system was retrained. Testing with the matching AI face (*womanface_test*) gave a correct

recognition with a clear margin (0.1316 against 0.1817 for the next closest), as shown in Figure 3.

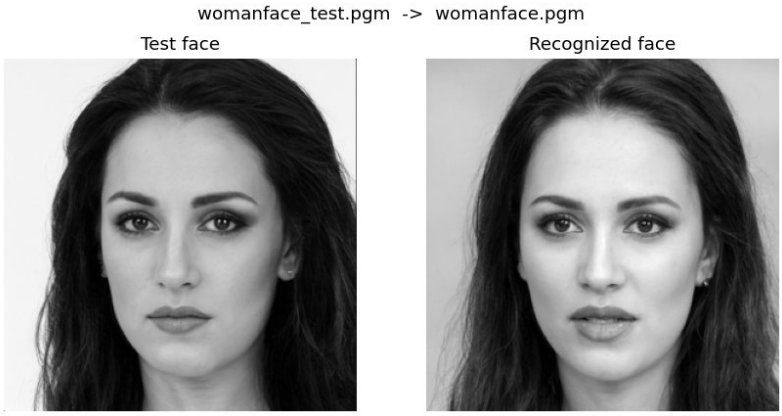

Figure 3. The newly added AI-generated face is recognised correctly.

Test image	Recognised as	Min. distance (x1e9)	Result
womanface_test	womanface	0.1316	Correct
girl_t	girl	0.1185	Correct
spanishwoman_t	spanishwoman	0.2307	Correct
brownwoman_t	brownwoman	0.2298	Correct (girl2 = 0.2303)
chinat_t	spanishwoman	0.3115	Wrong (chinawoman = 0.3120)
bread_t	beard	0.0091	Non-face input

Table 2. Recognition after adding the AI-generated face (eight training faces).

An interesting effect appears for *brownwoman_t*. With seven training faces it was wrongly matched to girl2 (0.1982 vs 0.2292 for the correct brownwoman). After adding a single new face, the decision flipped to the correct brownwoman (0.2298 vs 0.2303 for girl2). Both decisions are extremely close, which shows how sensitive the method is: adding one image changes the whole eigenface basis and therefore every distance.

Figure 4a. brownwoman_t with seven training faces — wrongly matched to girl2.

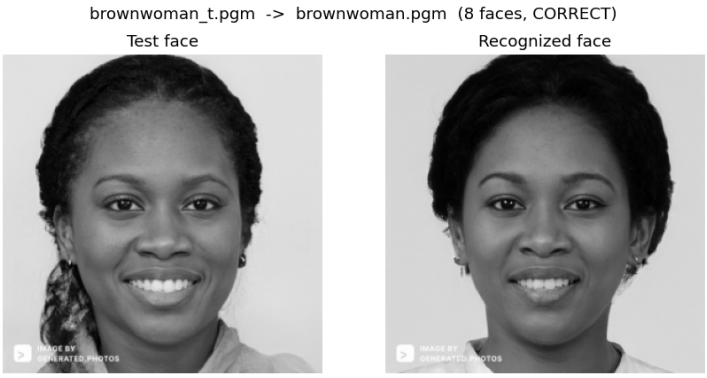

Figure 4b. brownwoman_t after adding the AI face — now correctly matched to brownwoman.

4.3 Real-time camera test

The webcam script detects the face in real time, tracks corner points on it and lets the user capture a 300×300 grayscale photo. A first photo was captured and added to the training set; a second photo, captured later under similar conditions, was then used as the test image. As shown in Figure 5, the captured test face was correctly matched to the captured training face, even though the person was wearing glasses and a headset and the lighting was dim. Face detection itself was initially unstable and became reliable only after removing the back-light and adding histogram equalisation and short-term box persistence to the tracking loop.

(In this public version the camera figure is omitted for privacy, as it shows the author's own photograph. The captured test face was correctly recognised as the author's training capture.)

5. Discussion

The Python implementation reproduces the MATLAB reference exactly, including the numerical distance values, which confirms that the algorithm is correct. Recognition quality, however, depends strongly on how similar the test image is to the training image of the same person.

For faces that are well aligned and similar to their training photo (girl, spanishwoman, the AI-generated face and the camera capture), recognition was reliable and the correct match had a clearly smaller distance. For harder cases such as brownwoman and chinat, the system failed or only barely succeeded, even though the same identities were present in training, because differences in expression, pose and lighting move the face far in the high-dimensional space. This is the main weakness of Eigenfaces: it is a holistic method that compares the whole image and is therefore very sensitive to imaging conditions.

Two further observations stand out. First, the non-face input *bread_t* was still assigned to its nearest neighbour with a very small distance: the system always returns the closest training face and has no way to reject an unknown or non-face input. A distance threshold would be needed for that. Second, the recognition is sensitive to the training set itself — adding a single face was enough to flip a borderline decision, because every training image reshapes the eigenface basis.

Overall, the Eigenfaces method is simple, fast and works well on frontal, uniformly lit, well-aligned faces — exactly the conditions of the provided dataset. It is far less robust when the test conditions differ from the training conditions, and it cannot, on its own, decide that an input is not a known face.

6. Conclusion

A working Eigenfaces face recognition system was implemented in Python and validated against the original MATLAB scripts, giving identical results. It correctly recognised most well-aligned faces, the added AI-generated face and the live camera face, while failing on a few harder cases and on non-face input. The experiments illustrate both the simplicity and the limitations of the Eigenfaces approach.